Computational Framework for Clinical Trial Simulation

Survival Analysis and ODE-Based Modelling for GBM Immunotherapy

Mariami Garuchava

Student ID: 01317059219

Degree: Bachelor of Science

Program: Computer Science

University: Kutaisi International University

Supervisor: Nino Demetrashvili

Date: June 2026

Abstract

Glioblastoma (GBM) is the most aggressive primary brain cancer, with a median survival of only 12 to 15 months despite standard surgery, radiotherapy, and chemotherapy. While immunotherapy holds promise, it has largely failed as a monotherapy in clinical trials, which motivates the study of combination regimens. Because real clinical trials are expensive, slow, and risky, this thesis presents a three-layer computational framework for in silico clinical trial simulation as a tool for early evaluation of treatment strategies. The system is based on the tumor-immune ODE model, which describes the interactions between glioma cells, activated T cells, and myeloid-derived suppressor cells (MDSCs) under two drug controls: anti-PD-1 and a CCR2 antagonist. The framework consists of a deterministic ODE core; a stochastic layer that generates a virtual-patient cohort of 1,000 individuals using log-normal Monte Carlo sampling to account for biological variety; and an analysis layer that produces Kaplan-Meier survival curves, log-rank tests, and Cox hazard ratios. To make the trial more realistic, the simulation includes responder heterogeneity and patient censoring. Benchmarked three numerical solvers - RK45, LSODA, and Radau are also presented to find the most efficient choice for large-scale simulations. The results from the 1,000-patient cohort show that combination therapy is the most effective treatment arm, followed by anti-PD-1 monotherapy and then the CCR2 antagonist. Combination therapy roughly halved the progression risk compared to no treatment $\left(\text{Hazard Ratio} = 0.52, p < 10^{-28}\right)$. Regarding computational performance, LSODA was the fastest solver, performing approximately twenty times faster than Radau because it can automatically switch strategies to handle the "stiffness" of the biological equations. This research demonstrates that in silico trials are a valuable and efficient way to optimize complex immunotherapy schedules before moving to human testing.

Keywords: glioblastoma, ODE modelling, Monte Carlo simulation, Kaplan-Meier, immunotherapy, scientific computing, RK45

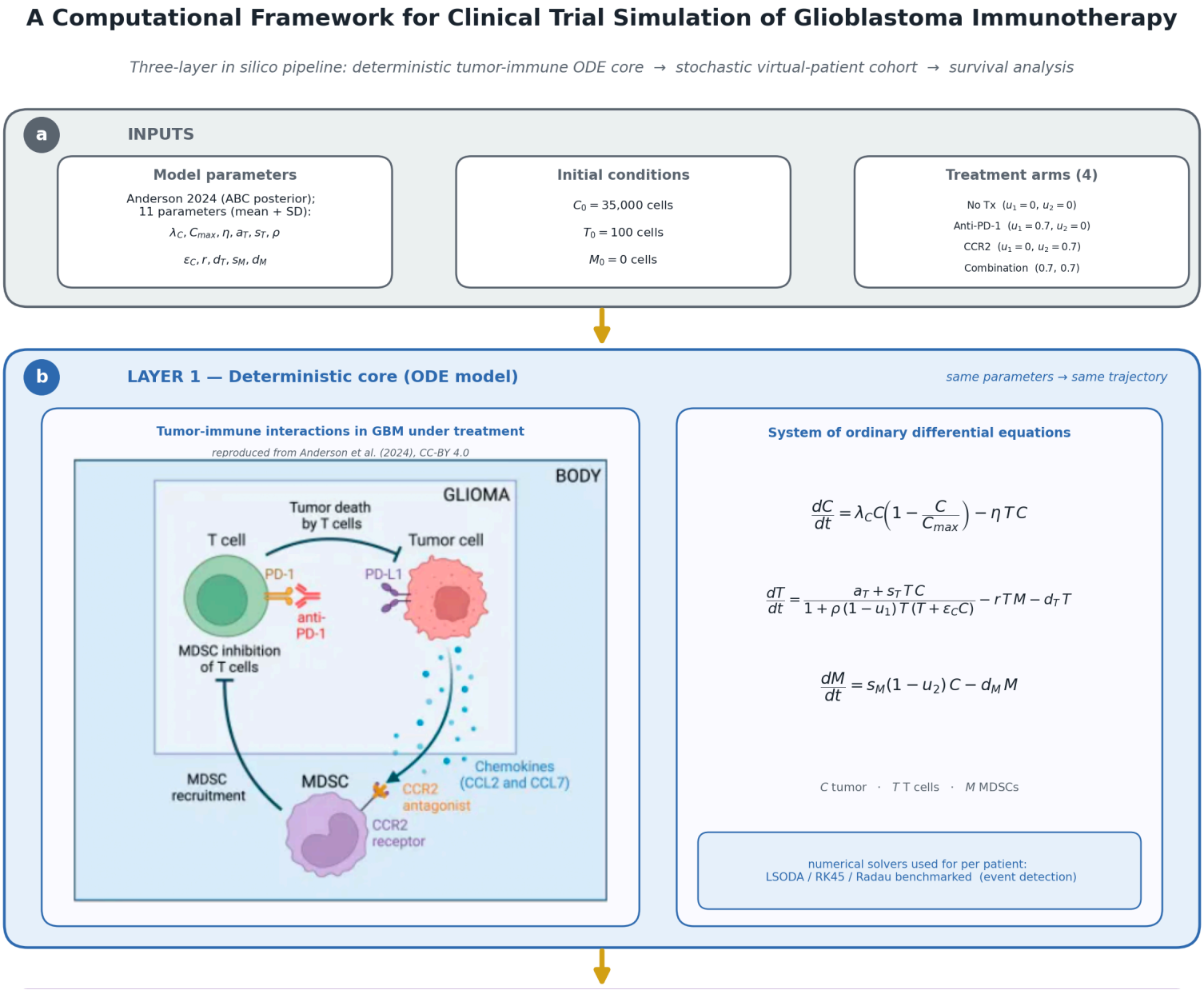
Table of Contents

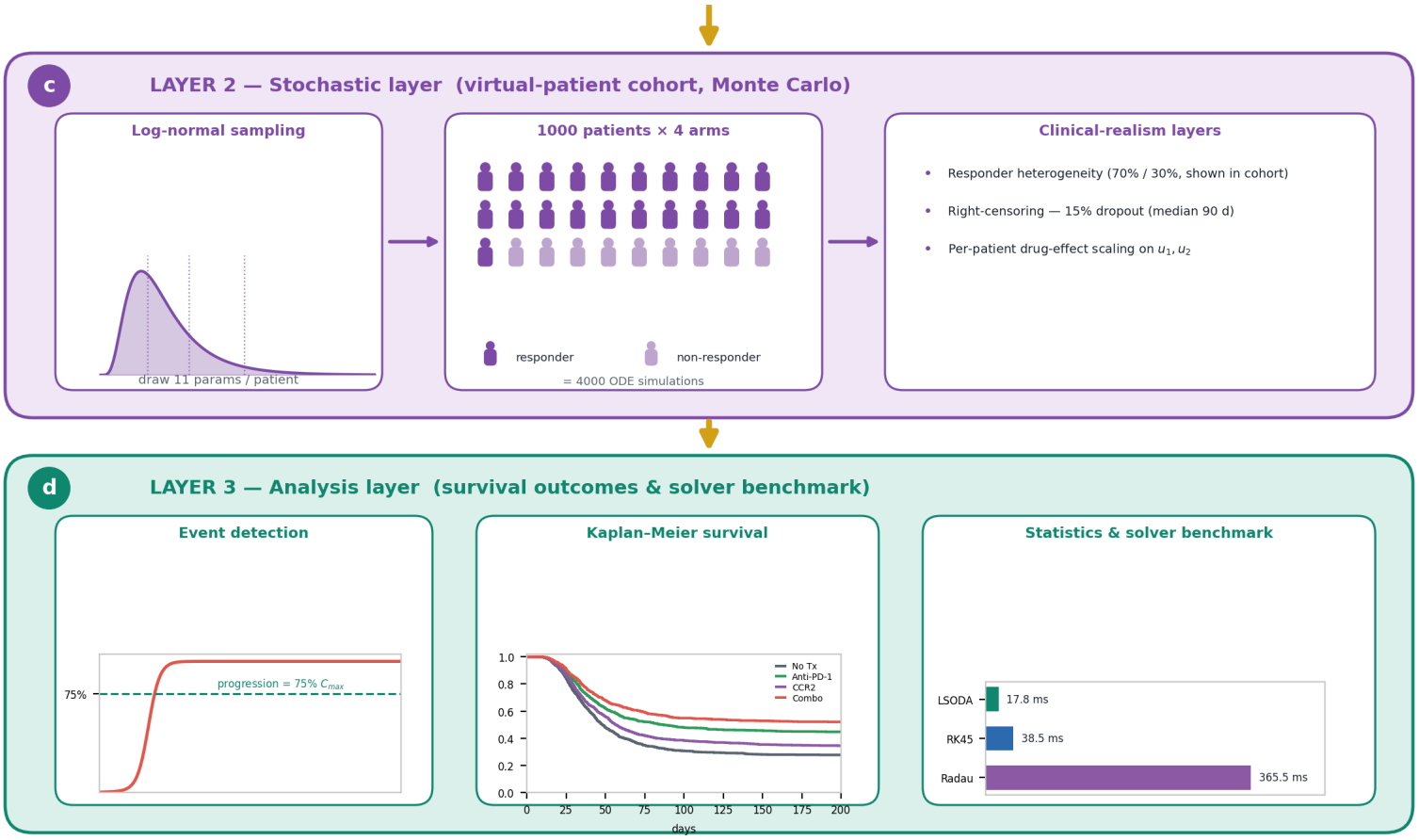
Abstract 1
1 Introduction 4
2 Biological background 8
2.1 Glioblastoma 8
2.2 The Blood-Brain Barrier 8
2.3 The Immune System and Cancer 9
2.4 Combination Immunotherapy 11
3 Mathematical Background 12
3.1 Ordinary Differential equations 12
3.1.1 State Variables and Parameters 12
3.1.2 Order of an equation 13
3.1.3 Initial Value Problem (IVP) 13
3.1.4 Linear and Nonlinear systems 13
3.1.5 Importance of numerical solution for ODE system 13
3.2 Numerical Foundations and Runge-Kutta Methods 14
3.2.1 Dormand & Prince method (RK45) 16
3.2.2 RADAU & LSODA numerical solvers 18
3.3 Monte Carlo simulation 18
3.3.1 History of Monte Carlo 19
3.3.2 main use cases of Monte Carlo 20
3.3.3 The Law of Large Numbers (LLN) 20
3.3.4 The Central Limit Theorem 21
3.3.5 Convergence rate 21
3.3.6 Sampling from distributions 21
3.3.7 Pseudorandom number generation (PRNG) 22
3.3.8 Pseudorandom number generator (PRNG) in clinical trials 22
3.3.9 The log-normal distribution 23
3.3.10 Application of log-normal distribution in clinical trials 23
3.4 Survival Analysis 23
3.4.1 Time-to-event data and censoring 23
3.4.2 The Kaplan-Meier estimator 24
3.4.3 The log-rank test 24

3.4.4 Hazard ratios 25
3.4.5 Median survival time 26
4 The GBM model 27
4.1 Source papers and modeling strategy 27
4.2 Model assumptions and simplifications 27
4.3 State variables 28
4.4 The glioma equation 28
4.5 The T cell equation 29
4.6 The MDSC equation 30
4.7 Drug control variables u_1 and u_2 31
4.8 Parameters 32
4.9 Initial conditions 34
4.10 Resting T cell Equilibrium T^* 35
4.11 Progression Endpoint 35
5 Implementation 36
5.1 Software and Tools 36
5.2 Layer 1 — ODE system 36
5.3 Layer 2 — Creating the Virtual Patient Cohort 37
5.4 Layer 3 — Analysis 37
5.5 Reproducibility and outputs 38
6 Results 39
6.1 Virtual-patient cohort 39
6.2 Tumour–immune dynamics 39
6.3 Survival outcomes 40
6.4 Solver efficiency 42
7 Conclusion 44
8 Appendix 45
8.1 Appendix A — GitHub Repository 45
References 46

1 Introduction

This thesis presents a 3-layered software system designed to solve an ODE system based on human parameters through Monte Carlo sampling. The system provides numerical solutions to identify the most effective treatment among four distinct arms for patients diagnosed with Glioblastoma, while simultaneously focusing on the progression-free survival (PFS) endpoint. Furthermore, it evaluates and identifies the most efficient numerical solver necessary for the high-speed sampling of a large virtual population. The figure below illustrates the core architectural design of the entire project.





Model: Anderson et al. (2024a, 2024b). Implementation: Python (scipy.integrate, scipy.optimize, lifelines). Endpoint: progression-free survival. Reproducible (seed = 42).

Background and Motivation

Glioblastoma (GBM) is the most common and aggressive primary brain cancer, characterized by a highly complex and immunosuppressed tumor microenvironment [1], [2]. Despite medical intervention, the median survival remains between 12 to 15 months, with a five-year survival rate of less than 10%. The current standard of care, known as the Stupp protocol - consisting of surgery, radiotherapy, and temozolomide (TMZ) - has remained largely unchanged since its establishment in 2005. While immunotherapy drugs exist, they have largely failed as monotherapies in Phase III clinical trials for GBM due to the tumor's advanced survival strategies. Because real-world clinical trials are expensive, slow, and risky, there is a critical need for alternative methods to evaluate new treatments. A viable solution lies in the computational simulation of clinical trials, which allows for the testing of various treatment arms and scenarios within a virtual environment.

Problem Statement

The primary research problem is: How can ODE-based models combined with Monte Carlo simulation be used to effectively simulate GBM immunotherapy clinical trials and predict patient outcomes? This approach seeks to bridge the gap between theoretical mathematical models and clinical application by creating a "numerical stochastic process" that mimics real-world patient variability.

Research Questions

Can the ODE system correctly model GBM-immune dynamics? The study investigates if the system accurately represents the interactions between glioma cells, T cells, and MDSCs.

Which ODE solver is most efficient for repeated Monte Carlo runs? This involves benchmarking different numerical methods, such as RK45, LSODA, and Radau, to determine which is best suited for large-scale simulations.

What does virtual patient simulation reveal about combination immunotherapy? The research examines if pairing treatments like anti-PD-1 and CCR2 antagonists provides a significant survival advantage over monotherapy.

Objectives

The thesis aims to achieve the following four specific objectives:

Implement the ODE system: Encode the mathematical equations representing glioma cells, activated T cells, and myeloid-derived suppressor cells (MDSCs).

Numerical methods comparison: Evaluate and compare the computational efficiency of different mathematical solvers for the ODE system.

Monte Carlo virtual patients: Generate a realistic virtual cohort by randomly sampling biological parameters from established distributions.

Kaplan-Meier survival analysis: Calculate survival statistics and visualize outcomes to compare the efficacy of different treatment arms.

Thesis Structure

The thesis is organized into several chapters to systematically address the research problem.

Chapter 2 explains human biology and the characteristics of the Glioblastoma.

Chapter 3 provides the mathematical background, covering ordinary differential equations, numerical methods, Monte Carlo sampling techniques, and the principles of survival analysis.

Chapter 4 presents the GBM model, detailing the source papers, biological assumptions, state variables, and parameter distributions used in the study.

Chapter 5 describes the implementation, outlining the three-layer software architecture - the ODE layer, the Monte Carlo cohort generation layer, and the analysis layer.

Chapter 6 details the results, including the virtual patient distributions, simulated tumor-immune dynamics, survival outcomes for the different treatment arms, and solver efficiency benchmarks.

Finally, Chapter 7 provides the conclusion, summarizing the main findings and suggesting directions for future research.

2 Biological background

2.1 Glioblastoma

Glioblastoma (GBM) is the most common and aggressive primary brain cancer. It is known as a highly malignant tumor that comprises primary brain malignancies. It creates a highly complex and immunosuppressed tumor microenvironment, which makes it exceptionally difficult to treat [1], [2]. Regarding its epidemiology, approximately 350,000 people are newly diagnosed with brain tumors globally each year, resulting in roughly 250,000 deaths. Most patients succumb to the tumor within 12 to 15 months of diagnosis, even with medical intervention. Within five years of diagnosis, the survival rate is less than 10%. The standard of care [2], known as the Stupp protocol, consists of surgical resection followed by radiotherapy and chemotherapy with temozolomide (TMZ). This protocol has remained the primary treatment strategy since it was established in 2005. Despite aggressive treatment, recurrence is essentially universal [1], [2]. Even when the tumor is significantly reduced, patients are highly likely to relapse regardless of how close they come to a "disease-free" condition.

The motivation to move toward immunotherapy stems from the minimally effective results of older standard treatments and the need for new strategies [1]. Immunotherapy is considered to be a good treatment strategy because its goal is to harness the immune system to overcome the tumor's defenses. While single-agent treatments like anti-PD-1 have failed as monotherapy in Phase III clinical trials, combination immunotherapies - such as pairing immune checkpoint inhibitors with therapeutics that target immunosuppressive myeloid-derived suppressor cells (MDSCs) - are being explored to increase the cytotoxic immune response and improve long-term survival outcomes [1], [2]. The biological mechanisms underlying this immunotherapy approach - the PD-L1/PD-1 immune checkpoint, MDSC-mediated suppression, and the reasons why combination therapy outperforms monotherapy - are explained in detail in Sections 2.4 through 2.6.

2.2 The Blood-Brain Barrier

The blood-brain barrier (BBB) [2] is a specialized system of microvascular endothelial cells that serves as the interface between the central nervous system (CNS) and the peripheral blood circulation. Anatomically, the BBB consists of endothelial cells

joined by tight junctions, along with pericytes and astrocyte end-feet, which create a highly selective semipermeable barrier. It evolved to protect the CNS from toxins, pathogens, and fluctuations in blood chemistry, maintaining a stable environment for neural signaling. This same protective mechanism makes drug delivery exceptionally hard because the BBB prevents approximately most small-molecule drugs and nearly all large-molecule drugs (like the anti-PD-1) from entering the brain at therapeutic concentrations [2]. While the BBB is a formidable obstacle, in the context of GBM, it is often locally disrupted. This is referred to as the "leaky BBB". In a growing tumor, the rapid, chaotic growth of new blood vessels (angiogenesis) often results in "leaky" vessels that are structurally weaker than normal brain capillaries, allowing some therapeutics to penetrate the tumor core more easily than they would healthy brain tissue [1], [2].

2.3 The Immune System and Cancer

The immune system's primary role in oncology is to identify and destroy malignant cells to prevent tumor progression. However, aggressive cancers like glioblastoma (GBM) create a highly complex and immune-suppressed environment that prevents the immune system from functioning effectively. Modern research into immunotherapy seeks to harness and amplify the body's natural immune defenses to overcome these barriers, though many monotherapies have failed because tumors develop advanced survival strategies [1], [2].

T cells are critical components of the adaptive immune response. In the study of GBM-immune dynamics, T cells are represented by all cells carrying the CD3+ marker, which includes both CD4+ helper T cells, which coordinate the immune response by releasing signaling molecules like cytokines to support other immune cells, and CD8+ cytotoxic T cells, often called "killer" T cells, the primary effectors that directly attack cancer cells. In these models, activated T cells refer to those that have been stimulated either by the presence of a tumor or by cytokines like IL-12, resulting in their recruitment and influx to the tumor site to perform their defensive duties [1], [2].

Activated T cells are considered the main drivers of tumor cell death. When these T cells reach the glioma microenvironment and interact directly with malignant cells, they kill them at a specific biological rate [1], [2].

The tumor microenvironment is the local environment surrounding a tumor, consisting of cancer cells, immune cells, blood vessels, and signaling molecules. In the

context of glioblastoma, the TME is notoriously immunosuppressive. Instead of being a site of active immune defense, the GBM microenvironment is often reprogrammed by the tumor to actively recruit cells that shield the cancer from T cell attacks and promote tumor progression through mechanisms like angiogenesis [1], [2].

Immune evasion is a general phenomenon where cancer cells develop the ability to mask themselves from recognition or actively downregulate the immune response. Cancers evade the immune system, effectively disguising the tumor cells and suppressing the cytotoxic response even when T cells are present. This evasion is often the primary reason why tumors continue to grow despite a patient having a technically functional immune system. GBM evades the immune system through two specific mechanisms: PD-L1 and MDSCs, which are covered in the next two sections [1], [2].

The PD-L1 / PD-1 immune checkpoint is a prominent axis of immunosuppression that glioblastoma (GBM) cells use to defend the immune system from attacking healthy cells. T cells have an " off switch, " which means the PD-1 (programmed cell death protein 1) receptor is an inhibitory receptor located on the surface of activated T cells, and when this switch is triggered, it sends signals to the T cell to stop attacking healthy cells. GBM cells produce a protein called PD-L1. When the tumor's PD-L1 connects with the T cell's PD-1 receptor, the tumor is being masked, making it invisible to the immune system. This interaction is the one that allows the tumor to grow, because it turns off the T cells that are supposed to kill the cancer. Drugs called anti-PD-1 are designed to block this interaction. By binding to the PD-1 receptor, the drug prevents the tumor's PD-L1 from connecting, which unmasks the tumor and lets the T cells do their job again. While these drugs work well for other cancers, they often fail when used alone (monotherapy) in GBM because these drugs only work if there are enough activated T cells already near the tumor to start an attack, but GBM often keeps T cell numbers very low, and, also, GBM uses other methods to hide, specifically MDSCs (myeloid-derived suppressor cells), which continue to shut down T cells even if the PD-1 "off switch" is blocked. Those are the main reasons why monotherapy fails [1], [2].

The CCR2 pathway is one of the primary ways glioblastoma (GBM) protects itself from the immune system. It involves a signaling loop where the tumor sends out chemokines to bring in myeloid-derived suppressor cells (MDSCs). By blocking this specific pathway, the influx of these cells might be stopped, and that will allow the immune system to attack the tumor more effectively. **MDSCs (myeloid-derived**

suppressor cells) are immature cells from the bone marrow. In a healthy body, these myeloid cells would eventually turn into regular immune cells. However, in the presence of cancer, they remain in an immature state and are "reprogrammed" to become highly suppressive. Instead of helping the body, they act as a shield for the tumor, shutting down the T cells that are trying to kill the cancer [1], [2].

GBM recruits CCL2/CCL7 chemokines. The recruitment process consists of three main steps. It starts with chemokine release: GBM cells produce and secrete signaling proteins called chemokines, specifically CCL2 and CCL7. Then these chemokines act as a beacon for any cell carrying a CCR2 receptor. And, after that, MDSCs express the CCR2 receptor on their surface, and when they detect the CCL2/CCL7 "scent," they travel from the bone marrow directly into the tumor microenvironment. CCR2 antagonists are drugs (such as CCX872 or BMS-813160) that bind to the receptor first, and by doing so, they prevent the tumor's chemokines from connecting, which keeps the MDSCs from ever reaching the brain [1], [2].

2.4 Combination Immunotherapy

In the context of glioblastoma (GBM), combination immunotherapy refers to the concurrent use of two different types of treatments to overcome the tumor's complex defenses. This specific research focuses on pairing anti-PD-1, an immune checkpoint inhibitor, with a CCR2 antagonist, which targets immunosuppressive cells. The goal is to maximize the body's natural immune response by removing multiple biological "brakes" (PD-1) at once. Previous chapters (2.4 & 2.5) describe why only using anti-PD-1 or CCR2 alone fails. However, combining these two therapies creates synergy because they target the two primary "axes" of GBM immunosuppression simultaneously. The CCR2 antagonist reduces MDSC levels, which indirectly increases the number of activated T cells available to fight the tumor, and once these T cell numbers are bolstered, the anti-PD-1 drug can more effectively "unmask" the tumor, allowing the larger number of T cells to recognize and destroy the malignant cells. The experimental evidence and biological synergy justify why clinical simulations and preclinical studies use four distinct treatment arms: No Treatment (Control), Anti-PD-1 Monotherapy, CCR2 Antagonist Monotherapy, Combination Therapy [1], [2].

3 Mathematical Background

For clinical trial simulation, the computational framework is built using a 3-layer architecture consisting of: **The ODE system**, **Monte Carlo Simulation**, and **Survival Analysis**.

This chapter explores all three layers in order to understand the core mechanisms underlying clinical trial simulations.

3.1 Ordinary Differential equations

Let f be a continuously differentiable function, a so-called "smooth" function. An *ordinary differential equation* (ODE) has the following form:

$$x' = f(t, x, \lambda) \tag{1}$$

Where the prime denotes differentiation with respect to the independent variable t (usually a measure of time). the dependent variable x is a *vector of state variables*, and λ is a *vector of parameters*.

The equation

$$m\frac{d^2x}{dt^2} + b\frac{dx}{dt} + kx = 0 \tag{2}$$

is an **ordinary differential equation**, because it involves only ordinary derivatives $\frac{dx}{dt}$ and $\frac{d^2x}{dt^2}$. That is, there is only one independent variable, the time t.
In contrast, the heat equation

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2} \tag{3}$$

is a **partial differential equation** — it has both time t and space x as independent variables.

3.1.1 State Variables and Parameters

State variables are a set of variables needed to fully describe a mathematical state of the dynamic system. x is a vector which is composed of all state variables. Table 3 shows appropriate state variables for the GBM model, which are elaborated in the following chapters.

λ is a vector composed of constants, and it is used to define the system's behavior. Parameters remain fixed during each simulation. They describe different kinds of physical or biological properties. The list of parameters used for the GBM model,

their descriptions, and units are shown in Table 4, and summary statistics are shown in Table 5.

Later chapters also discuss how the parameters were chosen during simulation.

3.1.2 Order of an equation

The highest-order derivative with respect to the independent variable determines the order of the equation. The GBM model consists only of first-order equations, since only first-order derivatives appear. (11) (12) (13)

3.1.3 Initial Value Problem (IVP)

The GBM model discussed in this paper uses differential equations to describe the evolution of state variables for a physical process. From the initial value of the state variable, its future values need to be determined. The mathematical model is then defined by a pair of equations:

$$x' = f(t, x, \lambda), \qquad x(t_0) = x_0 \tag{4}$$

The second equation is called an *initial condition* and the pair of equations is called an ***initial value problem***. Respectively, a solution is a function $x(t)$ that satisfies both the differential equation and the initial condition.

3.1.4 Linear and Nonlinear systems

A framework for ordinary differential equations is provided by the following form of system

$$\begin{aligned} x' &= f_1(x_1, ..., x_n) \\ &\vdots \\ x'_n &= f_{n(x_1,...,x_n)} \end{aligned} \tag{5}$$

This system is said to be **linear**, if all the x_i on the right-hand side have degree one. Otherwise, the system is **nonlinear**.

3.1.5 Importance of numerical solution for ODE system

Nonlinearity makes differential equation systems very difficult and most of the time, impossible, to solve analytically. The reason behind this is simple: The most important difference is that linear systems are easy to break down into several parts, solve them separately and recombine them in the end to get the answer. This simplifies the solving process of complex problems and forms the basis of some famous methods, such as Laplace transforms, superposition arguments, and Fourier analysis.

However, the principle of superposition fails for nonlinear systems, and that's where the numerical solution is needed.

3.2 Numerical Foundations and Runge-Kutta Methods

Before moving to the numerical implementation part, I first introduce and provide a brief explanation of some of numerical methods, starting from the simplest one.

Euler's method is the simplest numerical method for approximating the solution of an ODE. Given the *differential equation* $\dot{x} = f(x)$ with *initial condition* $x = x_0$ at $t = t_0$. Obviously, the goal is to find a systematic way to approximate the solution $x(t)$. Starting from x_0, the local velocity is $f(x_0)$. Over a short time, denoted as Δt, the phase point moves approximately $f(x_0)\Delta t$, which would only be an exact if the velocity is constant, a good approximation is achieved for small enough Δt. This gives a new position $x_1 = x_0 + f(x_0)\Delta t$. Iterating, the update rule becomes:

$x_{n+1} = x_n + f(x_n)\Delta t$. Euler's method can be improved since it estimates the derivative only at the left end of the time interval between t_n and t_{n+1}. A better approach would be to use the average derivative between across this interval. That is ***improved Euler's method***. While easy to implement, Euler's method requires a massive number of steps for high precision.

Local truncation error (LTE) defines a measure of the precision of the mathematical approximation used for a single step; basically, it is the difference between the true solution and the numerical solution after one iteration. **Global error** describes the difference between the true solution and the numerical solution at any given point.

Higher-order methods do not automatically mean that they are superior, because they require more calculations and function evaluations, which makes them computationally expensive.

Fourth-order Runge-Kutta (RK4) method tends to nicely balance computational efficiency and high accuracy. This method is a bit different from the actual one used in this paper. RK4 uses a fixed step size to solve ODE's. For a step size Δt, the update rule is:

with the four stage values defined as:

$$
\begin{aligned}
\boldsymbol{k}_1 &= \boldsymbol{f}(\boldsymbol{x}_n)\Delta t \\
\boldsymbol{k}_2 &= \boldsymbol{f}\left(\boldsymbol{x}_n + \frac{1}{2}\boldsymbol{k}_1\right)\Delta t \\
\boldsymbol{k}_3 &= \boldsymbol{f}\left(\boldsymbol{x}_n + \frac{1}{2}\boldsymbol{k}_2\right)\Delta t \\
\boldsymbol{k}_4 &= \boldsymbol{f}(\boldsymbol{x}_n + \boldsymbol{k}_3)\Delta t
\end{aligned}
\tag{6}
$$

[3]

The reason why RK4 is called fourth-order method is that it has local truncation error $\mathcal{O}(\Delta t^5)$ and global error $\mathcal{O}(\Delta t^4)$. A fixed step size of $\Delta t = 0.1$ typically provides sufficient accuracy for smooth low-dimensional dynamical systems.

See [3] and [4] for better understanding this chapter.

In this paper, the ODE system is solved by the numerical method named the Dormand & Prince method, but before actually discussing the implementation steps, we need to go through the historical development and mathematical analysis of higher-order Runge-Kutta (RK) numerical methods.

The primary goal of embedded Runge-Kutta formulas is to provide an efficient way to perform automatic step size control. Rather than using Richardson extrapolation, which requires computing two small steps and one large step to estimate error, embedded formulas construct two numerical approximations - y_1 (order p) and y_2 (order $\hat{p}$) that share the same function evaluations (k_i).

The method uses a single scheme of coefficients (c_i, a_{ij}) to compute a set of stages. These same stages are then combined using two different sets of weights (b_i, $\hat{b}_i$) to produce a lower-order solution (y_1) and a higher-order solution ($\hat{y}_1$).

$$
\begin{array}{c|cccc}
0 & & & & \\
c_2 & a_{21} & & & \\
c_3 & a_{31} & a_{32} & & \\
\vdots & \vdots & \vdots & \ddots & \\
c_s & a_{s1} & a_{s2} & \ldots & a_{s,s-1} \\
\hline
 & b_1 & b_2 & \ldots & b_s \\
\hline
 & \hat{b}_1 & \hat{b}_2 & \ldots & \hat{b}_s
\end{array}
\tag{7}
$$

Figure 3: Butcher tableau of an embedded Runge-Kutta method. By using a single scheme of coefficients (c_i, a_{ij}), the stages $k_1, ..., k_s$ are computed. Because these stages are shared, the two different numerical approximations (y_1 and $\hat{y}_1$) are produced using the same function evaluations. [5, Eq. (4.6)].

The lower-order approximation with an order of p is

$$y_1 = y_0 + h(b_1 k_1 + ... + b_s k_s) \tag{8}$$

and the higher-order approximation with an order of $\hat{p}$ is

$$\hat{y}_1 = y_0 + h\left(\hat{b}_1 k_1 + ... + \hat{b}_s k_s\right). \tag{9}$$

Estimate of the local truncation error is provided by the difference between these two results (y_1 - $\hat{y}_1$). Using the same function evaluations (k_i) for both approximations makes code more flexible and less computationally expensive

Fehlberg's embedded methods were developed in the 1960s, and his idea was to minimize the error coefficients of the lower-order result (y_1) in order to make his methods optimal.

The lower (4th) order approximation is intended to be used as an initial value for the next step, while the 5th-order result serves only as the error estimator. The lower-order approximation was intended to be used as an initial value for the next step.

In order to make his methods optimal, Fehlberg tried to minimize the error coefficients for the lower-order result (y_1) by using the *standard integration*, which means that it uses the lower (4th) order approximation and continues using it during integration, while the 5th-order result serves only as the error estimator.

The local extrapolation is different from the standard one; it continues integration using the higher-order result ($\hat{y}_1$) and chooses the step size by the difference between the two results.

Since Fehlberg uses the standard integration, it increases the accuracy of the 4th-order solution.

Its disadvantage is that it fails selecting step size, because Fehlberg definitely optimized y_1 and it someimes becomes almost as accurate as $\hat{y}_1$ and that can lead to substantially smaller estimated error than the true error ($\mathrm{E} = y_1 - \hat{y}_1$) because y_1 and $\hat{y}_1$ are too close to each other.

3.2.1 Dormand & Prince method (RK45)

RK45 is also an embedded Runge-Kutta pair of orders 5 and 4. Its numerical efficiency and special design distinguishes from the Fehlberg method. This method is used to solve the GBM ODE system described in this paper.

Unlike **Fehlberg's method**, which optimizes the lower-order solution, **Dormand & Prince** uses local extrapolation to advance integration using the higher (5th) order approximation.

Below, in the table, there are specially derived coefficients shown that are used to minimize the error coefficients of the 5th-order result.

0							
$\frac{1}{5}$	$\frac{1}{5}$						
$\frac{3}{10}$	$\frac{3}{40}$	$\frac{9}{40}$					
$\frac{4}{5}$	$\frac{44}{45}$	$-\frac{56}{15}$	$\frac{32}{9}$				
$\frac{8}{9}$	$\frac{19372}{6561}$	$-\frac{25360}{2187}$	$\frac{64448}{6561}$	$-\frac{212}{729}$			
1	$\frac{9017}{3168}$	$-\frac{355}{33}$	$\frac{46732}{5247}$	$\frac{49}{176}$	$-\frac{5103}{18656}$		
1	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$	
y_1	$\frac{35}{384}$	0	$\frac{500}{1113}$	$\frac{125}{192}$	$-\frac{2187}{6784}$	$\frac{11}{84}$	0
$\hat{y}_1$	$\frac{5179}{57600}$	0	$\frac{7571}{16695}$	$\frac{393}{640}$	$-\frac{92097}{339200}$	$\frac{187}{2100}$	$\frac{1}{40}$

Table 1: Butcher tableau of the Dormand–Prince 5(4) method (DOPRI5). In here, there are seven stage evaluations: $k_1, ..., k_7$, which are defined by column c and matrix A. Down below, there are two weight rows producing a fifth-order solution (y_1) which is used to determine a step size by local extrapolation, and a fourth-order solution ($\hat{y}_1$) only needed to estimate the error. Source: [5, Table 5.2].

First Same As Last (FSAL) property is a key innovation of the RK45 method. As shown above, Dormand & Prince have seven stage evaluations, and the 7th stage is identical to the 1st stage (k_1) of the subsequent step. This obviously reduces computational cost, because the method only needs to evaluate six functions per step instead of seven.

In this chapter, the main beneficial points of an embedded Runge-Kutta method - Dormand & Prince were discussed.

Summary of these key points is shown below on the Table 2.

RK45 (DOPRI5)	Pros
Higher precision	The coefficients are determined in a way that this method is significantly more precise than RKF45, Which implies a better numerical performance.
computationally efficient	FSAL property decreases computational cost
Local Extrapolation	Advances integration using the higher (5th) order approximation which guarantees efficiency
dense output	provides precise approximations and graphical output at any point between steps without additional function evaluations

Table 2: Key advantages of Dormand & Prince method

If you need further details and proofs for the methods provided in this chapter, see [5].

3.2.2 RADAU & LSODA numerical solvers

In implementation, two other numerical solvers are used too with RK45 to compare their efficiency. **LSODA** [6] (Livermore Solver for Ordinary Differential Equations, with Automatic Method Switching) is a high-precision numerical solver designed to solve systems of ODEs. Its defining feature is its ability to automatically choose between different integration methods based on the state of the system. By monitoring data in real-time, it can decide whether a problem is stiff or non-stiff and uses different methods for these two cases. See [6] for more information about LSODA numerical solver.

RADAU [7] refers to a class of implicit Runge-Kutta methods used for the numerical solution of differential equations. It has high-precision method with excellent numerical stability. See [7] for further details of RADAU.

A **non-stiff system** is one where all variables change at a similar pace, allowing the solver to take large steps easily. A **stiff system** occurs when some biological processes happen much faster than others. This forces simple solvers to take tiny, time-consuming steps to avoid 'crashing' or becoming unstable, which is why specialized solvers like Radau or LSODA are necessary to handle those fast-changing dynamics efficiently.

3.3 Monte Carlo simulation

The Monte Carlo method uses random sampling to solve mathematical problems.

3.3.1 History of Monte Carlo

The first experiment, which used random sampling to find a solution for a mathematical problem, was documented in 1977 by a French mathematician, Comte de Buffon. He was trying to calculate the probability that a thrown needle onto a horizontal plane would have an intersection with aligned straight lines on that plane. If length of the needle is $\boldsymbol{L}$, and the straight lines are distanced by $\boldsymbol{d}$, where $d > L$, then the probability $\mathbf{P}$ that randomly thrown needle will intersact with one of these lines is given by this formula:

$$\boldsymbol{P = \frac{2L}{\pi d}} \tag{10}$$

Buffon derived this formula by performing an experiment of throwing a needle onto the plane many times.

That method was not used as a general computational tool for centuries, but some scientists found it captivating. For instance, Laplace advanced Buffon's experiment by using a grid instead of just straight lines. Then, Lord Kelvin used random sampling for kinetic theory. He was randomly drawing papers with numbers on them from a bowl. In the 1930s, while Enrico Fermi was working on the behavior of the newly discovered neutron, he used random sampling experiments to understand how neutrons interact with matter.

The formal name was given in the 1940s by scientists who were working on nuclear weapons at Los Alamos. One of the scientists, Stanislaw Ulam, saw the potential of that method while he was playing solitaire (a game where you arrange cards into specific groups), and trying to estimate the chance of winning the game by simulating the playing many times, he thought that was the easiest solution, and that led him to use that tool in nuclear physics. In 1949, he and Nicholas Metropolis first described the Monte Carlo method in their published article in *Journal of the American Statistical Association.* See [8].

For more historical background of Monte Carlo, see [9].

Formally, Monte Carlo is a technique for solving a mathematical problem.

through random sampling, when an analytical calculation gets too complex.

The Monte Carlo method is **stochastic** because it relies on random sampling. In stochastic processes, each state is determined by random events. By Monte Carlo, we can find the solution of non-probabilistic problems using probabilistic methods, and simulation is the direct transcription of a natural random process. The development

of digital computers has significantly increased the efficiency of Monte Carlo methods, since it is much easier to run simulations on a computer rather than just perform the experiment in real life.

3.3.2 main use cases of Monte Carlo

The two primary use cases for the Monte Carlo method involve its application as a numerical tool for solving mathematical problems and as a direct model for complex, real-world random processes.

For clinical trial, we are basically **simulating stochastic systems**, meaning, we use Monte Carlo to mimic a system which has random behaviour.

A stochastic process is defined as a sequence of states where the evolution from one state to the next is determined by random events. When we perform a clinical trial simulation using Monte Carlo methods, we are creating a "numerical stochastic process" on a computer to mimic real-world variability.

Key aspects of why clinical trial simulations are considered stochastic:

Modeling Random Events: Clinical trials are often modeled as sequences of random events, For instance, the provided GBM model in this paper has clinical endpoint which estimates time from treatment initiation until disease starts to progress - Progression-Free Survival (PFS). That uncertainty is modeled by random numbers.

Discrete Event Simulation: This specific type of Monte Carlo approach is used to test systems that inherit stochastic behaviour. It generates artificial random events to see how a system, like a clinical trial, performs under different conditions.

Handling Medical Data: In medicine, simulation is used to model realistic scenarios that traditional analytical math struggles to solve easily, such as survival times for patients or comparing the success rates of different treatments.

Realistic Distributions: Older - "standard" models used simplified, often non-realistic distributions. Stochastic simulations have the ability to use more complex and realistic models that better represent the actual variability found in human health data.

3.3.3 The Law of Large Numbers (LLN)

The Monte Carlo method works because it transforms non-probabilistic problems, such as multidimensional integrals, into a stochastic structure that can be solved

through random sampling, and the Law of Large Numbers (LLN) provides the mathematical justification for why Monte Carlo provides a valid numerical estimate.

The Law of Large Numbers states that if you take a sequence of independent and identically distributed random variables $X_1, X_2, ..., X_n$, their sample mean $\overline{X}_n$ will converge to the true population mean μ as the number of samples n approaches infinity. That means, the more times we perform an experiment, the average result will go closer and closer to the true expected value.

3.3.4 The Central Limit Theorem

The Central Limit Theorem is used to place confidence intervals around Monte Carlo estimates, which means it provides a precise way to measure how reliable the simulation results are. While the Law of Large Numbers guarantees that an estimate will eventually converge to the true value, the **Central Limit Theorem** is used to describe the distribution of the error that occurs during that process.

The Central Limit Theorem states that the distribution of a Monte Carlo estimator will behave as a normal distribution for a large enough sample size (N). To be able to apply Central Limit Theorem, the original distribution needs to provide a **finite variance**.

If the variance is infinite (like the Cauchy distribution, which is a special case known as having undefined variance), the limit distribution will not be normal, meaning that this method can not be used to calculate standard confidence intervals.

Although even if the finite variance exists, the method will only guarantee highly accurate results as $N \to \infty$, because if the sample size N is too small, the central limit theorem might derive optimistic confidence intervals [9].

3.3.5 Convergence rate

Because of the LLN, as the sample size N increases, the variance of the mean value decreases at a rate of $\frac{1}{N}$. This means the probability of a large deviation between the Monte Carlo estimate and the true value becomes very small as more trials are conducted. While the variance decreases as $\frac{1}{N}$, the statistical/standard error is defined as the square root of the variance, it shrinks as $\frac{1}{\sqrt{N}}$.

See [9].

3.3.6 Sampling from distributions

Sampling is the core of the Monte Carlo method, and its capability is to produce a sequence of random variables that follow a specific probability distribution function.

3.3.7 Pseudorandom number generation (PRNG)

All computational Monte Carlo sampling fundamentally starts with uniform random variables on the interval (0,1), which are generated by a computer as **pseudorandom numbers** because all other distributions require a sequence of uniform variables to be simulated. See [10].

Pseudorandom number generation produces sequences of numbers that mimic the statistical properties of truly independent and identically distributed random variables. However, because these sequences start from an initial value - **seed**, they are not truly random but are reproducible, meaning, if the same seed is used all the time, the same sequence of numbers will be generated See [10], [9].

Mersenne Twister is modern algorithm to generate uniform random variables. It features a period of $2^{19937} - 1$, ensuring that the sequence does not repeat even during the most extensive computational simulations. It can generate sequences that maintain statistical randomness across a very high number of dimensions because of a 623-dimensionally equidistributed generator.

Like other deterministic algorithms, it starts from an initial value - a seed. Using a fixed seed enables replaying an exact simulation, which is critical for debugging and regulatory validation in fields like clinical trials. Those are the reasons why the Mersenne Twister still remains the default.

pseudorandom number generator in MATLAB, R, and Python's standard library.

Implementation based on that paper for the GBM system uses more recent algorithm **Permuted Congruential Generator** (PCG-64) [11], it is the default in NumPy's modern `default_rng` (recommended function that initializes a random number generator) API [12].

Like the Mersenne Twister, PCG-64 is deterministic and seeded.

3.3.8 Pseudorandom number generator (PRNG) in clinical trials

In the context of clinical trials, a pseudorandom number generator (PRNG) is an essential tool used for the construction of stochastic models that represent the inherent variability and uncertainty found in patients.

populations. Some of its benefits are the following: ***virtual patient cohorts can be generated*** by drawing human characteristics from specific distributions, ensuring that virtual patients exhibit realistic interindividual variability. Since clinical trials are stochastic systems, PRNG can be used for ***modeling Stochastic event***, it

mimics natural random process within trials. By performing many Monte Carlo simulations, different scenarios can be tested, that ***optimizes trial design and power***. Simulations are useful for ***handling non-standard scenarios*** - If a trial is expected to violate traditional statistical assumptions, such as the Proportional Hazard Assumption (the ratio of the risks between the two groups remains constant over time), which is common in cancer immunotherapy trials where survival benefits may be delayed. For more precise information about this section, see [13] and [9].

3.3.9 The log-normal distribution

The log-normal distribution is considered to be the optimal choice for modeling parameters in clinical and statistical simulations because its range is strictly restricted to positive values $(0, \infty)$. A log-normal variable Y is produced by taking the exponential of X, where X is a normal variable: $\boldsymbol{Y = e^X}$. Since the exponential of any real number is always non-negative, that guarantees the positivity of the parameters, which is a necessary requirement for biological parameters [10].

3.3.10 Application of log-normal distribution in clinical trials

Another benefit of using a log-normal distribution for patient health data is that it can **handle Skewness**. Human health data often exhibits a "right-skewed" behavior where most observations are clustered at lower values, but a few "long-tail" observations exist at very high values. The log-normal distribution naturally captures this skewness, making it a more faithful representation of real patient data than a symmetric Normal distribution. See more on [9], [10].

3.4 Survival Analysis

Survival analysis is a specialized branch of statistics used to analyze **time-to-event data**, which measures the duration of time until a specific event occurs. In clinical oncology, the event is often **overall survival** (OS), which defines the time from treatment until a patient's death, or the clinical endpoint of the GBM model presented in this paper - **Progression-Free Survival (PFS)**, which measures the time before the tumor starts to grow again.

3.4.1 Time-to-event data and censoring

In a simulation, the **"Event time"** is determined by the "disease trajectory" of a virtual patient. For instance, if the model simulates tumor growing until it reaches a pre-defined point (when the tumor can not grow more than it is during that time of simulation), the "death" event occurs, and that elapsed time is "Event time".

Time-to-event data is a specific statistical data type consisting of the event times and censoring status recorded across the entire virtual cohort. See [14].

The censoring is the primary characteristic that makes

Time-to-event data is unique and different from standard statistical methods. It might occur when a patient's actual event time can no longer be recorded because the study ended [10].

In such cases, it is only known that the survival time was greater than the duration of the observation.

The standard way to represent censored data models is: $(\boldsymbol{T_i}, \boldsymbol{\delta_i})$ where T_i is observed time and δ_i - a binary indicator. T_i can be derived from the following formula: $T_i = \min\{Y_i, \overline{u}\}$ where Y_i is the event time and $\overline{u}$ is the duration of the study. The binary indicator stores the information if the event was actually observed ($\delta_i = 1$) or if the observation was censored ($\delta_i = 0$).

Traditional analytical math struggles to accurately process such cases without specialized survival analysis models [10], [14].

3.4.2 The Kaplan-Meier estimator

One specific model is the Kaplan-Meier estimator, which visualizes survival curves from time-to-event data while, at the same time, is able to handle censoring [14]. It is the standard method for visualizing survival outcomes in clinical research or major cancer immunotherapy trials [14]. The Kaplan-Meier provides a non-parametric estimate of survival probability for some time intervals, meaning it shows the actual patterns of survival, even if they are unusual, and the estimator calculates the probability of surviving for each time interval (for instance, every month). By multiplying these probabilities together, it builds a survival curve that drops whenever an event (like a death) occurs [10], [14].

3.4.3 The log-rank test

The log-rank test is used as a fundamental statistical tool in survival analysis. It compares the time-to-event data between two or more treatment groups, known as arms [14]. The primary use of the log-rank test in clinical research is to determine if there is a statistically significant difference between the survival distributions of different cohorts.

The log-rank test is also used for determining the efficacy of various dose regimens, such as comparing a treatment arm, which might be immunotherapy, against a

control arm, like an arm where patients are on a placebo or are receiving a standard of care [14].

Null Hypothesis (H_0) states that there is no difference between the survival distributions of the cohorts being compared. This implies that the hazard ratio (see precise definition in the following subsection) is equal to 1, meaning the risk that the event occurs is the same in all treatment arms. **Alternative Hypothesis** (H_1) asserts that the survival distributions differ statistically from each other. In that case, the hazard ratio is not equal to 1, indicating that one treatment regimen provides a survival advantage/disadvantage over the other. In clinical trial simulations, the test produces a **p-value**. The trial is considered to be a "positive" if $p < 0.05$, which represents the Significance Threshold [14].

The log-rank test handles censoring properly because it does not rely on knowing the final event time for every patient. Instead, it performs comparisons based on risk sets at each discrete event time. Clinical trial data used in simulations include risk tables, which record the number of patients that are considered to be "at risk" of an event at various time points.

Comparative mechanism of the test is following: At every time an event occurs, the test compares the number of observed events in a treatment group to the number expected if the null hypothesis were true. Because it only considers patients who have neither had the event nor been censored yet at that specific moment, it incorporates the information from censored patients for exactly as long as they were observed [10], [14]. When the virtual cohort is partitioned into multiple arms, the log-rank test compares the survival distributions across these arms simultaneously, detecting whether

They differ significantly. See more about survival analysis in [15]. The four arms defined for the GBM model and corresponding survival analysis are described in the implementation chapter.

3.4.4 Hazard ratios

A hazard ratio (HR) is a statistical measure that indicates the difference in risk of an event (like a death) between two treatment groups over time. It is derived from the hazard function, which represents the instantaneous event rate at a specific time t, given that the subject has survived up to that point [15].

The hazard ratio between a treatment arm (A) and a reference arm (B) is defined as: $\boldsymbol{HR = \frac{\lambda_A(t)}{\lambda_B(t)}}$ and it provides a direct comparison of risk. If

$HR = 1$, that is the indicator of the equivalent risk, meaning there is no observed difference in the hazard rates between the two arms.

$HR < 1$ indicates reduced risk in the treatment arm relative to the reference, while $HR > 1$ implies elevated risk in the treatment arm [15].

3.4.5 Median survival time

The median survival time is a fundamental summary measure in survival analysis, defined as the shortest time at which the survival function is ≤ 0.5. It represents the time when it is expected that half of the cohort has experienced the event. The method for determining the median survival time depends on the type of data and the statistical model being used.

From a Kaplan-Meier curve ($\hat{S}(t)$), the median is the time at which the estimated survival probability drops to or below 0.5. This method is used for the GBM model, which is described in the implementation chapter.

The median is often preferred over the mean survival time for several reasons. As mentioned earlier, human survival data is usually highly skewed, with a few long-term survivors creating a "long tail". The median provides a more representative "center" for such data than the mean and guarantees robustness to skewness. The median can also handle censoring effectively because calculating the mean requires conditions that do not always meet the clinical trial properties. Since computing the mean requires the survival curve to drop to zero (the entire virtual population experiences a death event), which rarely happens in clinical trials, if the curve does not reach zero due to censoring, the mean can only be "restricted" to a specific time window, whereas the median can still be calculated as long as the curve drops below 0.5 [15].

In clinical trials, particularly for cancer immunotherapy, median survival is a critical endpoint for comparing treatment arms. For the defined GBM model in this paper, the primary clinical endpoint is the median PFS for every four arms, which is computed from the corresponding Kaplan-Meier estimate. The following chapter also describes all these implementation details.

If you need further details and proofs for the methods provided in this chapter, see [5].

4 The GBM model

In this chapter, the GBM model is presented which is a combination of biology (chapter 2) and mathematics (chapter 3).

4.1 Source papers and modeling strategy

The mathematical model used in this simulation is derived from the work of Anderson et al. (2024a) [1] and Anderson et al. (2024b) [2]. Anderson et al. (2024a), published in the Journal of Mathematical Biology, establishes the underlying baseline dynamics and the original tumor-immune ODE system for **glioma cells (C)** (see Equation 11), **activated T cells (T)** (see Equation 12), and **myeloid-derived suppressor cells (M)** (see Equation 13). This foundational study also provides the parameter distributions (See Table 4 & Table 5). The model is further developed in Anderson et al. (2024b) [2], published in the Journal of Theoretical Biology, which extends the system by adding the drug control terms $\boldsymbol{u_1}$ (representing anti-PD-1) and $\boldsymbol{u_2}$ (CCR2 antagonist). While the 2024a paper provides the core biological framework, the drug-control equations used for optimizing treatment regimens are described in the 2024b paper.

4.2 Model assumptions and simplifications

The mathematical model for glioblastoma (GBM) dynamics is built upon several assumptions and simplifications to ensure practical identifiability and computational feasibility.

Spatial homogeneity: The model is a system of ordinary differential equations (ODEs), which assumes that all cell populations are well-mixed and lacks any spatial structure. Relaxing this assumption would require using partial differential equations (PDEs) or agent-based models to track cell migration, spatial gradients of chemokines, and localized tumor architecture.

BBB handled implicitly: The blood-brain barrier (BBB) is not modeled as a separate compartment. Instead, its restrictive filtering effect is absorbed directly into the constant drug control variables (u_1 and u_2), which represent the net therapeutic strength that successfully reaches the tumor microenvironment. Relaxing this assumption would necessitate a multi-compartment system to explicitly model the pharmacokinetic transport of therapeutics across endothelial tight junctions into the central nervous system.

Murine calibration: The model parameters are fit to preclinical experimental data from mouse models (KR158 glioma cells), meaning the biological rates may not perfectly reflect human oncology. Relaxing this would require re-estimating parameters like tumor growth rates and immune kill rates using human clinical trial data to account for different biological timescales.

Single sub-population per cell type: The system assumes that all glioma cells share a single set of parameters, ignoring the clonal heterogeneity typically found in aggressive brain cancers. Relaxing this would require additional equations to represent distinct tumor clones, such as those that are treatment-resistant or have different growth dynamics.

Constant doses: This simulation scope simplifies the time-varying optimal control paths ($u_1(t), u_2(t)$) described in the Anderson papers into fixed dosages for the treatment arms. Relaxing this would involve implementing the continuous administration schedules determined by optimization software to find the most efficient drug delivery paths.

4.3 State variables

The following state variables represent the core biological populations in the glioblastoma-immune dynamics model.

State variables	Biological meaning	Units
$C(t)$	Number of glioma (tumor) cells	cells
$T(t)$	Number of activated T cells (CD3+, includes CD4+ and CD8+)	cells
$M(t)$	Number of MDSCs (myeloid-derived suppressor cells)	cells

Table 3: State variables of the GBM tumor-immune ODE model (Anderson et al., 2024a).

4.4 The glioma equation

Based on the model developed in Anderson et al. (2024b), the differential equation representing the rate of change for glioma cells is:

$$\frac{dC}{dt} = \underbrace{\lambda_C C\left(1 - \frac{C}{C_{\max}}\right)}_{\text{logistic growth}} - \underbrace{\eta T C}_{\text{killed by T cells}} \tag{11}$$

The first term represents the **logistic growth** of the malignancy, where λ_c is defined as the intrinsic tumor cell growth rate. This parameter dictates how aggressively the cancer cells multiply over time when resources are abundant. This growth rate is a primary driver of tumor burden, and its interaction with other biological parameters determines the overall progression of the disease.

The variable $C_{\max}$ represents **the carrying capacity** of the tumor cells, serving as the biological "ceiling" for the population within the brain. It represents the maximum number of cells the environment can support based on physical space and nutrient availability. It is important to note that $C_{\max}$ is a mathematical limit of the tumor's size and is not the same as the experimental humane endpoint. The second term represents the mass-action **killing of tumor cells by activated T cells**. This model assumes that the destruction of cancer cells is dependent on the direct interaction between the effector T cell population (T) and the glioma cell population C. As T cells and tumor cells increase, the frequency of these encounters rises, leading to a higher overall rate of tumor cell death. The parameter $\boldsymbol{\eta}$ is the specific **kill rate** of tumor cells by T cells. It quantifies the efficiency of the body's natural defense mechanism, representing the speed at which an individual activated T cell can identify and eliminate a malignant cell. Mathematical analysis suggests that for a patient to reach a "tumor-free" state, this immune-mediated kill rate must be potent enough to overcome the tumor's intrinsic growth rate (λ_c) and the various forms of immunosuppression present in the microenvironment [1], [2].

4.5 The T cell equation

This differential equation representing the rate of change for activated T cells (T) is:

$$\frac{dT}{dt} = \left(\underbrace{a_T}_{\text{activation}} + \underbrace{s_T T C}_{\text{stimulation}} \right) \cdot \underbrace{\frac{1}{1 + \rho(T + \varepsilon_C C)(1 - u_1(t))T}}_{\text{inhibition by PD-1-PD-L1}} - \underbrace{rTM}_{\text{inhibition by MDSCs}} - \underbrace{d_T T}_{\text{death}} \tag{12}$$

The numerator $\boldsymbol{a_T + s_T T C}$ describes the two primary ways activated T cells enter or increase within the tumor microenvironment. The parameter a_T represents a constant homeostatic production or activation rate, occurring independently of the tumor's presence, often attributed to baseline immune signaling from cytokines like IL-12. This is combined with the $s_T TC$ term, which represents tumor-driven

stimulation. This models how the presence of glioma cells actively recruits more T cells and stimulates their proliferation directly at the tumor site.

The denominator $\frac{1}{1+\rho(T+\varepsilon_C C)(1-u_1(t))T}$ represents the PD-L1/PD-1 immune checkpoint inhibition, a mechanism of immune evasion where the tumor "exhausts" or turns off incoming T cells. The variable $u_1(t)$ serves as the anti-PD-1 control variable, representing the percentage of this inhibitory "off switch" that is blocked by the drug. The factor $\mathbf{(1-u_1)}$ mathematically weakens the inhibition in the model; as the drug efficacy reaches 100% ($u_1 = 1$), the inhibitory term in the denominator vanishes, allowing the T cell population to expand without these biological "brakes".

The term rTM models the suppression of T cells by myeloid-derived suppressor cells (MDSCs). The parameter r represents the specific suppression rate, quantifying how effectively the MDSC "bodyguards" can deactivate the cytotoxic immune response to protect the malignancy.

The final term, $-\ d_T T$, accounts for the natural death of T cells at a constant rate d_T. This term ensures the model remains biologically realistic by acknowledging that immune cells have a finite lifespan and will eventually succumb to natural apoptosis regardless of tumor interactions. In a healthy, tumor-free state, the steady-state level of the immune system is determined by the balance between this natural death rate and the baseline activation rate a_T [1], [2].

4.6 The MDSC equation

Based on the model developed in Anderson et al. (2024b), the differential equation representing the rate of change for myeloid-derived suppressor cells (M) is:

$$\frac{dM}{dt} = \underbrace{s_M C(1-u_2)}_{\text{stimulation by chemokines}} - \underbrace{d_M M}_{\text{death}} \tag{13}$$

The first term - $s_M C(1-u_2)$ represents the recruitment of MDSCs to the tumor microenvironment in response to signaling proteins (chemokines) secreted by the malignancy. Specifically, glioma cells produce CCL2 and CCL7, which act as a beacon for MDSCs carrying the CCR2 receptor. The recruitment rate is proportional to the number of tumor cells ©, as a larger tumor burden produces a higher concentration of chemokines. The variable u_2 serves as the CCR2 antagonist control, representing the percent reduction of this recruitment rate by the drug. By including the factor $(1-u_2)$,the model simulates how the therapeutic blocks CCR2 receptors to scale down

the influx of these cells; a theoretical value of $u_2 = 1$ would represent a complete blockade where no new MDSCs can reach the brain.

While the original optimal control framework in Anderson et al. (2024b) [2] utilizes time-varying functions $u_2(t)$ to determine mathematically ideal treatment schedules, this thesis simplifies the drug controls u_1 and u_2 to fixed constant values (e.g., 0.7) for the duration of the trial. This means the immunotherapy is at a constant therapeutic strength throughout the entire simulation. The primary aim of this research is not to determine an optimal dosing schedule, but rather to evaluate how different treatment arms compare in survival outcomes across a virtual cohort. For this comparison, a fixed representative dose per arm is the appropriate tool.

The final term, $d_M M$, accounts for the natural death of MDSCs at a constant rate d_M. This term ensures that the MDSC population remains biologically realistic by acknowledging that these cells have a finite lifespan. In these simulations, that internal clearance rate (d_M) is a critical parameter; subjects with higher natural MDSC death rates are more likely to achieve long-term survival, as their bodies can more effectively clear the immunosuppressive cells that protect the tumor.

4.7 Drug control variables u_1 and u_2

The variable u_1 represents the percent reduction of the PD-L1-PD-1 inhibition rate (ρ) by the administration of an anti-PD-1 immune checkpoint inhibitor. Biologically, this control simulates the drug binding to the PD-1 receptors on activated T cells, which prevents them from being "turned off" by the tumor's PD-L1 markers. By reducing this inhibition, the therapy effectively "unmasks" the malignancy and allows for a more robust cytotoxic immune response.

The variable (u_2) represents the percent reduction of the MDSC recruitment rate (s_M) by a CCR2 antagonist. This control simulates the therapeutic blocking of CCR2 receptors on myeloid-derived suppressor cells, which prevents them from detecting the CCL2 and CCL7 chemokines secreted by the tumor. By stopping these cells from ever reaching the brain, the drug reduces the overall immunosuppressive shield protecting the tumor.

Both control variables are bounded within the interval $[0, 1]$ to remain biologically and numerically consistent. $u_i = 0$ (No Drug) : This limit case represents a state where no treatment is administered, and the biological processes of immune inhibition or MDSC recruitment occur at their full, natural rates. $u_i = 1$ (Perfect Blockade): This represents a theoretical "perfect" treatment where the target mechanism is 100%

blocked. Because a perfect blockade is physiologically unrealistic, these simulations utilize a maximum effective value (such as 0.7 as implemented in code) to represent a more plausible therapeutic limit.

While the original Anderson papers utilize time-varying Lebesgue integrable functions ($u_i(t)$) to find optimal treatment paths, this paper treats these controls as constants as a primary modeling simplification (as noted in §3.2). Mathematically, the u_1 variable enters multiplicatively as the factor $(1 - u_1)$ inside the inhibition term of the T-cell denominator. This effectively relaxes the PD-L1/PD-1 checkpoint, allowing the T-cell population to expand more freely as the drug concentration increases. In the MDSC Equation, the variable u_2 enters multiplicatively as the factor $(1 - u_2)$ within the recruitment term $(sM \cdot C)$. This directly reduces the recruitment of new MDSCs from the bone marrow to the tumor site based on the efficacy of the antagonist.

4.8 Parameters

The model parameters were primarily established using the Approximate Bayesian Computation (ABC) rejection method as detailed in Anderson et al. (2024a) [1] and Anderson et al. (2024b) [2]. This computational approach produced posterior probability distributions for the biological rates by sampling one million parameter sets and accepting those that closely matched murine experimental data. Summary statistics for these parameters, including the best fit, mean, median, mode, and standard deviation, are provided for the glioma, T cell, and MDSC populations. While the source material often utilizes "Best Fit" values for representative subject simulations, this thesis uses the **Mean as the cohort center** and the **SD as the spread** to define the parameter ranges for the virtual population.

Parameter	Definition	Units
Glioma Cells (C)		
λ_C	Tumor cell growth rate	day^{-1}
$C_{\max}$	Carrying capacity of tumor cells	cell
η	Kill rate of tumor cells by T cells	day^{-1} cell^{-1}
T Cells (T)		
a_T	T cell activation rate	cell day^{-1}
s_T	Rate of tumor cell-mediated proliferation of T cells	day^{-1} cell^{-1}
ρ	T cell inhibition rate by PD-L1-PD-1 complex formation	cell^{-2}
ε_C	Expression of PD-L1 in tumor cells vs. T cells	—
r	T cell inhibition rate by MDSCs	day^{-1} cell^{-1}
d_T	T cell death rate	day^{-1}
Myeloid-Derived Suppressor Cells (M)		
s_M	MDSC recruitment rate by tumor production of chemokines	day^{-1}
d_M	MDSC death rate	day^{-1}

Table 4: Model parameters of the GBM–immune dynamics model: definitions and units.

Parameter	Best Fit	Mean	Median	Mode	SD
Glioma Cells (C)					
λ_C	0.431	0.272	0.250	0.174	0.114
$C_{\max}$	3.04×10^6	1.87×10^7	1.45×10^7	4.19×10^6	1.48×10^7
η	2.57×10^{-8}	1.84×10^{-7}	1.27×10^{-7}	4.03×10^{-8}	1.86×10^{-7}
T Cells (T)					
a_T	3.26×10^6	2.48×10^6	2.45×10^6	1.98×10^6	1.45×10^6
s_T	8.56×10^6	5.73×10^6	5.91×10^6	8.79×10^6	2.63×10^6
ρ	0.107	0.223	0.207	0.0768	0.142
ε_C	16.2	37.5	31.1	7.00	29.3
r	6.92×10^{-6}	2.85×10^{-5}	1.83×10^{-5}	3.60×10^{-6}	2.76×10^{-5}
d_T	0.0221	0.402	0.415	0.656	0.213
Myeloid-Derived Suppressor Cells (M)					
s_M	0.0466	0.0476	0.0476	0.0392	0.0040
d_M	0.251	0.258	0.263	0.419	0.143

Table 5: Posterior statistics of GBM model parameters from ABC rejection method (Anderson et al., 2024). Summary statistics of s_M re-estimated with the updated MDSC equation.

4.9 Initial conditions

The following initial conditions are used for all simulations, as established in Anderson et al. (2024b):

$\boldsymbol{C_0 = 35,000}$ **cells** Represents the initial orthotopic implantation of KR158 glioma cells into the murine brain. $\boldsymbol{T_0 = 100}$ **cells** Represents a small baseline population of activated T cells present at the tumor site at the start of the simulation. $\boldsymbol{M_0 = 0}$**cells** Reflects the biological reality that myeloid-derived suppressor cells are absent from the brain prior to the tumor's secretion of recruitment chemokines. It is important to note that the value of $T_0 = 100$ is a modeling choice by the authors based on experimental data and is not the resting equilibrium (T^*) of the immune system.

4.10 Resting T cell Equilibrium T^*

T^* is defined as the steady state of the immune system in a tumor-free environment ($C = 0$). Under these conditions, the T cell population dynamics are formed by the equation $\frac{a_T}{1+\rho T^2} - d_T T = 0$. Determining T^* requires solving a cubic root-finding problem, which this research performs numerically (as detailed in the Implementation chapter using the *brentq* algorithm). For the mean parameter set, this equilibrium is approximately $T^* \approx 302$ cells. Critically, T^* is an analysis quantity used to characterize the baseline strength of the host's immune system and it is not used as an initial condition for the tumor-progression simulations.

4.11 Progression Endpoint

The progression event in this simulation is defined as the point at which a subject's tumor cell count reaches 75% of that specific patient's own carrying capacity ($C_{\max}$). This threshold is chosen as a progression proxy for the subsequent survival analysis. This choice is necessitated by the parameter regime identified in Anderson et al. (2024b) [2], while a theoretical humane endpoint of approximately 2×10^8 cells is often discussed, the estimated biological ceiling for glioblastoma in these models ($C_{\max} \approx 1.87 \times 10^7$ cells) typically lies well below that value. Consequently, this thesis uses 75% of patient-specific ($C_{\max}$) to provide a consistent and realistic measure of treatment failure and disease advancement.

5 Implementation

This chapter describes the software implementation of the GBM clinical-trial simulation defined in Chapter 4. The framework has three layers where the first layer represents the ODE system described in the GBM model chapter, the second layer generates a virtual-patient cohort via Monte Carlo, and the third, analysis layer produces survival outcomes. This chapter describes the design of each layer, and the complete source code is provided in Section 8.1.

5.1 Software and Tools

The simulation is implemented in Python 3, which uses the following libraries: **NumPy, SciPy, lifelines, pandas, Matplotlib**. It has a deterministic behaviour with a *random seed 42*.

5.2 Layer 1 — ODE system

The first layer encodes the ODE system from Chapter 3 (Equation 11, Equation 12 Equation 13) as a single Python function. Given the current populations of tumor cells, T cells, and MDSCs, it returns their instantaneous rates of change. The function takes two drug controls u_1 and u_2 as constant scalars, and the 11 model parameters (Table 4). Since the goal of this thesis is just to compare treatments, and not to find the perfect dosing schedule, keeping drug strength fixed (at 0.7) is a reasonable design choice, and time-varying controls are left for future work. Before the simulation starts, the resting T-cell state T^* is computed. Because this fixed point is the root of a cubic equation, it cannot be written in a simple closed form, and is solved numerically. For the mean parameter set, this yields $T^* \approx 302$ cells. The initial condition for the simulations is $T_0 = 100$ cells. The simulation runs last for up to 200 days for each patient. Progression is defined at the moment when the tumor grows to 75% of the patient's maximum carrying capacity. To catch exactly when this happens, an "event function" is used, which is a built-in feature of the ODE solver. It is highly accurate and makes the code run faster because it stops calculating as soon as the limit is hit instead of just checking the tumor size every day or week, which would only give us an estimate. If 200 days pass and the tumor never reaches that size, then the patient is recorded as a survivor of the trial without progression.

5.3 Layer 2 — Creating the Virtual Patient Cohort

The basic model only represents one specific patient. To make it realistic, a cohort of virtual patients is needed. For this, Monte Carlo sampling is used, which basically means randomly generating different biological parameters for each person to match them with the natural differences in real glioblastoma patients. For every virtual patient, 11 different biological parameters are generated randomly by using a log-normal distribution. That method only produces positive numbers, which is important since biological parameters can not be negative. These 11 values are generated independently, using the averages and spreads from Table 5. Using just one single "best fit" number wouldn't produce the variety, so that's the reason why the "Mean" is taken as the center, and "SD" as the spread. To make the simulation clinically realistic, two more rules were added. First, 30% of our virtual patients are labeled as "non-responders," meaning the drug barely works on them. This reflects the reality that immunotherapy does not work for everyone. Second, 15% of the patients were randomly made to leave the trial early, which marks them as "censored" (see Section 3.4.1 for the definition).

One single group of 1000 virtual patients is generated, and all four different treatments are tested on this exact same group, so no new groups are created for each test. This results in non-random survivals because any difference in survival is purely because of the drug itself. The entire cohort, consisting of 1000 patients going through 4 treatments, means the code runs 4000 separate simulations in total.

5.4 Layer 3 — Analysis

The final step is turning raw data into actual results. For every virtual patient in every treatment group, the exact time when their tumor progressed, or if they dropped out early, is known. Now, survival statistics need to be calculated.

For each treatment, **Kaplan–Meier survival curve** is created. This is just the standard medical way to graph how many patients stay progression-free over time, and it can also automatically handle the "censored" patients, who dropped out. If more than half the people make it to the end of the trial without their tumor growing, the median survival time is reported as "not reached".

To see if a drug actually worked better than no treatment, The **log-rank test** and The **Cox proportional-hazards model** are run. The log-rank test can tell if the difference between these survival curves is random or actually real, while the Cox

proportional-hazards model provides the information about how much the treatment lowered the risk of the tumor growing (a number below 1 means the drug helped).

5.5 Reproducibility and outputs

When you run the main code, it goes through all three layers and saves everything to your computer. It generates the survival graphs, the raw data for every patient, and simple CSV tables with the final stats and numerical solver speeds.

A separate, smaller script is made just for drawing the graphs. This way, you can customize the appearance of the graph without running the entire heavy simulation again. A fixed "seed" for the random numbers makes the code reproducible. If you run it again, you will get the exact same patients and the exact same survival results. The only thing that changes is the millisecond timing of the solvers, since that depends on the hardware of the computer.

6 Results

This chapter covers the results of the clinical trial simulation. Everything provided here comes from one single, reproducible run of the code (using seed 42).

6.1 Virtual-patient cohort

To build our group of virtual patients, the code randomly picked 11 biological parameters for each person, using the log-normal distribution explained in Section 5. Figure 4 shows the numbers across the whole group. The graphs lean a bit to the right; that is just how log-normal distributions look. It highlights just how different real patients can be. For example, things like the tumor's maximum capacity ($C_{\text{“max”}}$) and the immune system's kill rate (η) can vary hugely from person to person. Because of this realistic variety of parameters, the realistic spread of survival outcomes is produced.

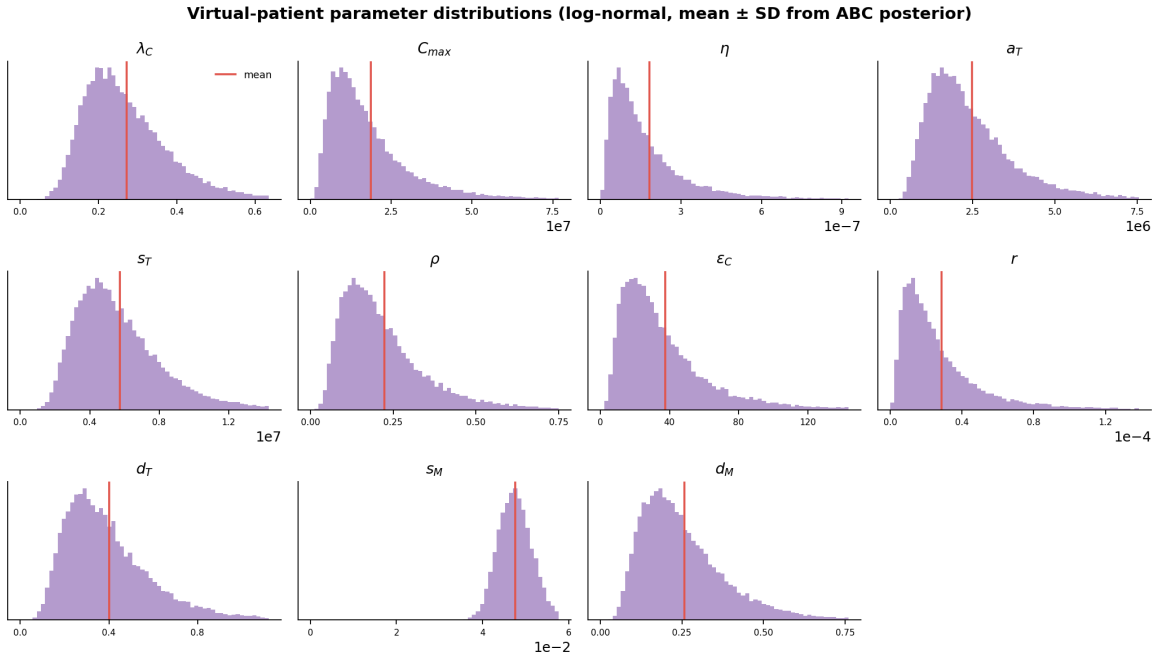


Figure 4: Distributions of the 11 model parameters across the virtual cohort. Each parameter is sampled from a log-normal distribution matching the mean (red line) and standard deviation

6.2 Tumour–immune dynamics

Figure 5 shows what happens inside a single, average patient over time under the four different arms.

When we give the patient **no treatment**, the tumor just keeps growing until it hits its maximum size, which takes about five weeks. T cells can not fight the tumor because they die, and MDSC cells help the tumor to grow. Under **anti-PD-1**, T cells get an opportunity to multiply and fight the tumor. When the patient is on the **CCR2 antagonist**, the MDSC cells are successfully neutralized, but without enough T cell help, the tumor still grows a bit. Under **combination therapy**, both drugs are given to the patient. The T cells stay high, the MDSCs drop to zero, and the tumor gets shut down . Using just one drug always leaves the tumor half-protected, which is why neither drug can completely stop the cancer on its own. Based on Anderson et al. ([2]): the combination therapy is the most effective, followed by anti-PD-1, then the CCR2 antagonist, and finally no treatment at all.

The "gaps" between the survival rates are smaller in this project. The reason is simple: Anderson et al. used complex, time-varying doses ($u_1(t)$ and $u_2(t)$) that were calculated to be the mathematically perfect schedule. However, in this case, the provided implementation is simplified by using constant doses. Since this GBM model isn't using an "optimized" schedule, the survival differences between the groups are a bit smaller.

In a way, our studies are answering different questions using the same model. Anderson et al. were looking for the best possible outcome you could ever achieve, while this paper just compares how the treatments perform against each other at a fixed dose.

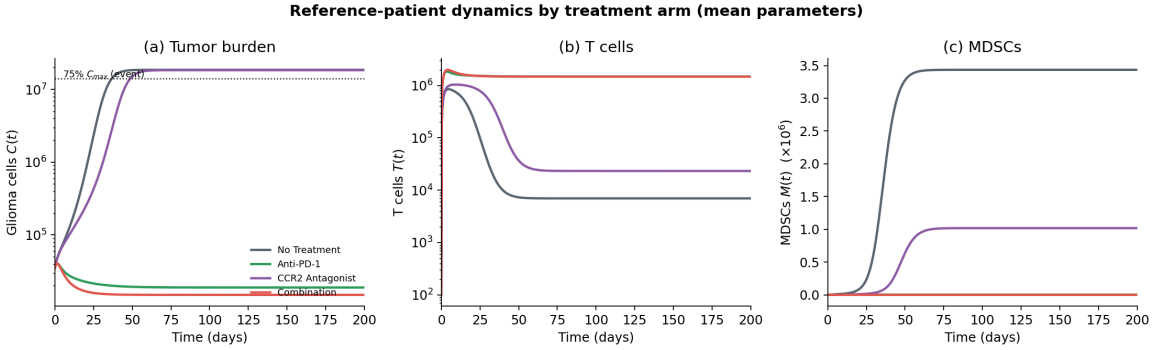


Figure 5: Reference-patient dynamics of glioma cells, T cells, and MDSCs over 200 days under each treatment arm (mean parameters). Tumour and T-cell axes are logarithmic.

6.3 Survival outcomes

Kaplan-Meier survival curves based on the entire cohort are shown on the Figure 6. The four treatments split apart clearly. The patients who were on the "No treatment"

arm result worst, with only about 28% of patients surviving without their tumor growing. The CCR2 drug helps a little bit, anti-PD-1 helps a lot, and the combination is the clear winner, with about 52% of patients staying progression-free by day 200.

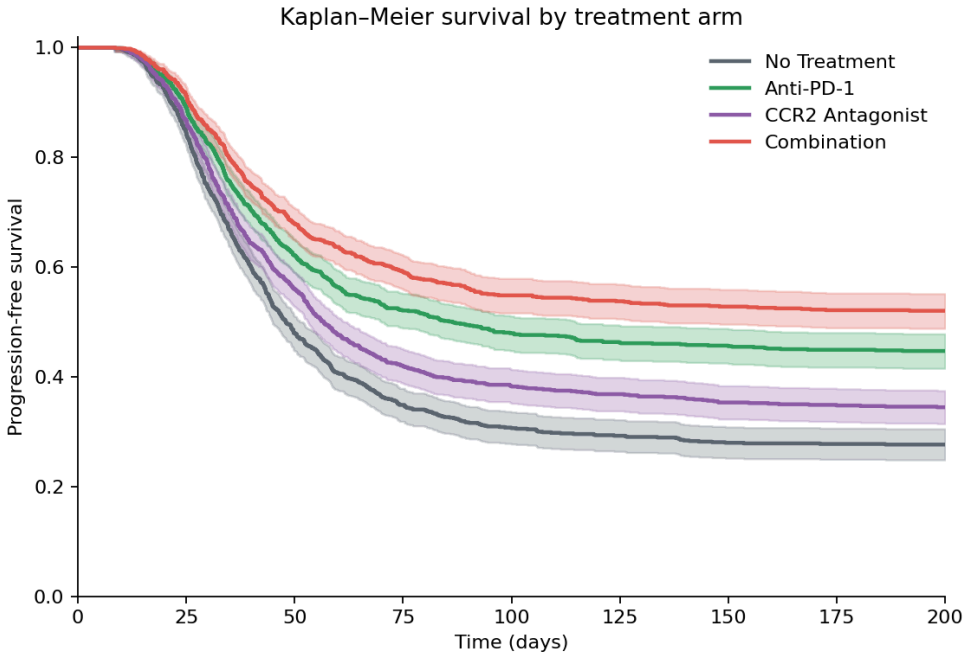


Figure 6: Kaplan-Meier progression-free survival by treatment arm, with 95% confidence bands. The shaded regions are the Greenwood confidence intervals.

The exact numbers for each treatment are in Table 6. The "hazard ratio" basically shows how much the drug lowered the risk of the tumor growing. The combination therapy cuts the risk in half compared to no treatment, while the CCR2 drug only gives a small benefit. According to the log-rank test, all of these improvements are statistically real, not just random.

Arm	**Events**	**Median PFS**	**PFS@200d**	**Log-rank p**	**Hazard ratio**
No Treatment	699	48.1 d	0.28	—	—
CCR2 Antagonist	632	56.7 d	0.34	5.7×10^{-4}	0.83
Anti-PD-1	533	86.4 d	0.45	2.5×10^{-15}	0.64
Combination	462	not reached	0.52	3.3×10^{-29}	0.52

Table 6: Per-arm survival statistics over 1000 patients. Hazard ratios are relative to No Treatment; values below 1 indicate benefit.

Visualization of these hazard ratios is shown on the forest plot in Figure 7, and the final survival counts in Figure 8. Based on these analytical results, we can confidently say the drugs actually worked, with the combination therapy working the best.

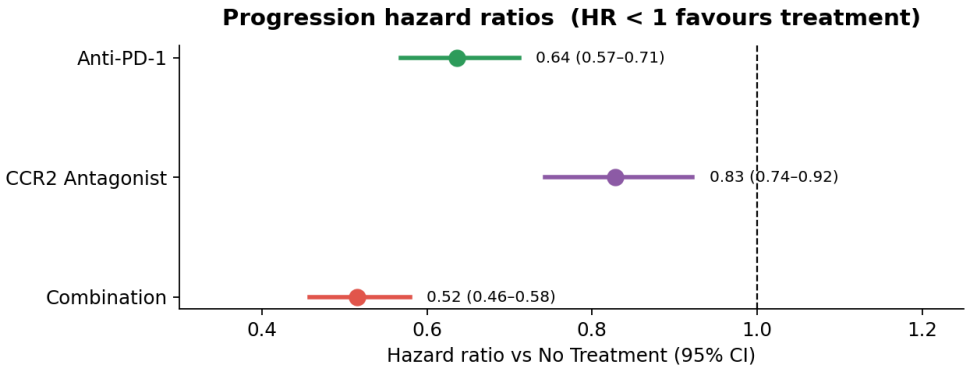


Figure 7: Hazard ratios for each treatment arm relative to no treatment, with 95% confidence intervals. All intervals lie below 1.

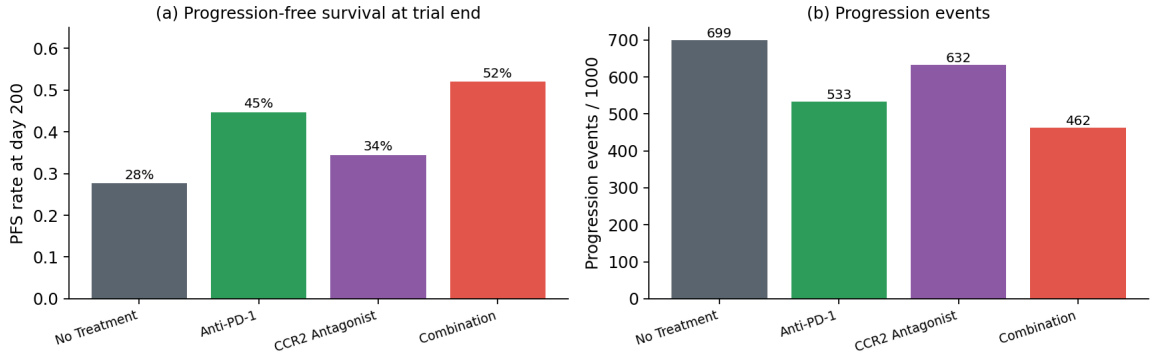


Figure 8: Progression-free survival rate at day 200 (left) and total progression events (right) by treatment arm.

6.4 Solver efficiency

Figure 9 compares efficiency of the three math solvers during a run on a single patient. LSODA is the clear winner, followed by RK45, while the Radau solver is about ten times slower. LSODA wins because of the fact that the tumor cells and immune cells grow at very different speeds. As mentioned earlier, LSODA is powerful because of its ability to automatically switch its math strategy since it can tell if the data is stiff or non-stiff; that's how it saves a ton of computing time. RK45 is an explicit method best suited to non-stiff problems, and is forced into very small step sizes whenever the system becomes stiff. Radau is an implicit method designed for stiff problems, but highly expensive because each step requires solving a system of equations, which is wasteful when the system is non-stiff. LSODA wins by combining both strategies.

The clinical simulation needs to be run a high number of times to derive a more reliable probability. So picking the fastest solver guarantees computational efficiency.

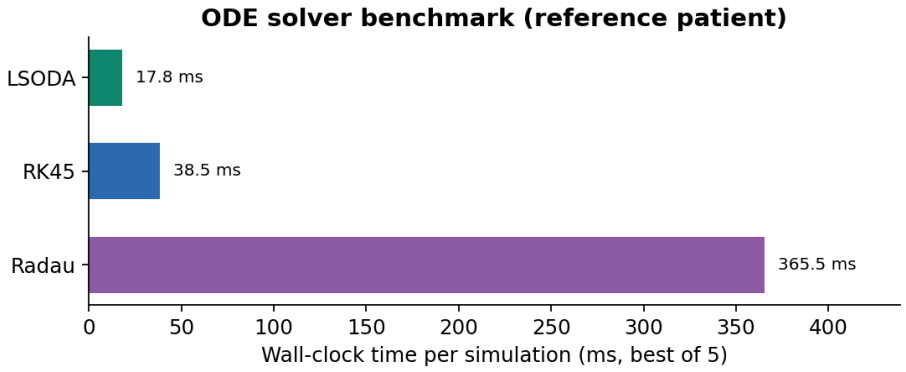


Figure 9: Wall-clock time per simulation for the three ODE solvers on a reference patient (best of five runs). Absolute times are machine-dependent; the ordering is stable.

7 Conclusion

This thesis presents a computer simulation for a virtual clinical trial testing glioblastoma immunotherapy and successfully answers the three main questions for which the entire project was built. The project takes its main inspiration from the tumor-immune math model created by Anderson et al. [1], [2]. The implemented code is organized into three main layers: the ODE system, creating the virtual patient cohort, and calculating the final survival statistics. The cohort with 1000 patients goes through four different treatments and is also tested with three different numerical solvers to find the most efficient one. The final results show that the model actually behaves like real biology under all four treatments (Figure 5). LSODA is definitely the most efficient solver for this kind of system, making it the perfect choice when you need to run thousands of simulations (Figure 9). Finally, combination therapy gives the best survival results, followed by anti-PD-1, and then the CCR2 drug (Figure 6 Table 6). Combination therapy being superior over other treatment groups was also one of the findings of Anderson et al.'s original paper [2]. However, it is possible to advance this study in several ways.

For future work, the most obvious next step would be to stop using constant drug doses and switch to the optimized, time-varying dosing schedule. Making that change should widen the survival gaps between the treatments and result in the absolute best possible outcome the model predicts these therapies can achieve.

8 Appendix

8.1 Appendix A — GitHub Repository

References

[1] H. G. Anderson, G. P. Takacs, D. C. Harris, Y. Kuang, J. K. Harrison, and T. L. Stepien, "Global stability and parameter analysis reinforce therapeutic targets of PD-L1-PD-1 and MDSCs for glioblastoma," *Journal of Mathematical Biology*, vol. 88, no. 1, p. 10, 2024, doi: 10.1007/s00285-023-02027-y.

[2] H. G. Anderson, G. P. Takacs, J. K. Harrison, L. Rong, and T. L. Stepien, "Optimal control of combination immunotherapy for a virtual murine cohort in a glioblastoma-immune dynamics model," *Journal of Theoretical Biology*, vol. 595, p. 111951, 2024, doi: 10.1016/j.jtbi.2024.111951.

[3] S. H. Strogatz, *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*, 2nd ed. Boca Raton, FL: CRC Press, 2015.

[4] C. Chicone, *Ordinary Differential Equations with Applications*, vol. 34. in Texts in Applied Mathematics, vol. 34. New York: Springer-Verlag, 1999.

[5] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, Second Revised Edition., vol. 8. in Springer Series in Computational Mathematics, vol. 8. Berlin, Heidelberg: Springer-Verlag, 1993. doi: 10.1007/978-3-540-78862-1.

[6] I. Misra and V. Ramanathan, "Belousov–Zhabotinsky reaction: an open-source approach," *Proceedings of the Indian National Science Academy*, vol. 88, pp. 243–249, 2022, doi: 10.1007/s43538-022-00081-6.

[7] J. Chen *et al.*, "Physics-informed neural networks for solving high-index differential-algebraic equation systems based on Radau methods," *International Journal of Intelligent Systems*, vol. 2024, p. 6641674, 2024, doi: 10.1155/2024/6641674.

[8] N. Metropolis and S. Ulam, "The Monte Carlo Method," *Journal of the American Statistical Association*, vol. 44, no. 247, pp. 335–341, 1949, doi: 10.1080/01621459.1949.10483310.

[9] M. H. Kalos and P. A. Whitlock, *Monte Carlo Methods*, 2nd ed. Wiley-VCH, 2008.

[10] C. P. Robert and G. Casella, *Monte Carlo Statistical Methods*, 2nd ed. in Springer Texts in Statistics. New York: Springer, 2004.

[11] M. E. O'Neill, "PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation," technical report HMC-CS-2014-0905, 2014. [Online]. Available: https://www.pcg-random.org/pdf/hmc-cs-2014-0905.pdf

[12] C. R. Harris and others, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[13] A. Bedding *et al.*, "Clinical Trial Simulations – an Essential Tool in Drug Development," London, UK, Position paper, Aug. 2014. [Online]. Available: https://www.abpi.org.uk/media/idkpteac/trial_design.pdf

[14] J. H. A. Creemers *et al.*, "In silico cancer immunotherapy trials uncover the consequences of therapy-specific response patterns for clinical trial design and outcome," *Nature Communications*, vol. 14, p. 2348, 2023, doi: 10.1038/s41467-023-37933-8.

[15] J. P. Klein and M. L. Moeschberger, *Survival Analysis: Techniques for Censored and Truncated Data*, 2nd ed. in Statistics for Biology and Health. New York: Springer, 2003.